Faculty of Engineering - Ain Shams University

Online Marketplace System: Phase I Report

CSE 352: Parallel and Distributed Systems

Name	ID
Arsany Hany Anwar	2200655
Andrew Ehab Tharwat	2200249
Hazem Mohamed Salah ElDin	2300774
Maria Karam Albert	2200593
Ziad Ahmed Orabi	2200399
Marten Amgad Zakaria	2200751
Mathew Mina Youssef Aziz	2300910
Kerlos Melad Fakhry	2200763
Julia Nasser Mosaad	2200760

Table of Contents

1. Introduction 2
2. Target Benefeciaries 3
3. Adopted Programming Language 4
4. System Architecture 5
   - Overview 5
     - High Level Architecture Diagram 6
     - Detailed Backend Layout 7
   - Internal Modules 8
     - Socket Server 8
     - Bridge Server 8
     - Spring Boot REST Server 9
     - React Frontend 9
     - Shared Models 10
     - Three Features Implemented as REST (Bonus) 11
   - API for Interfacing External Stores 13
     - Overview 13
     - Authentication 13
     - Endpoints 13
     - What Happens When an Order is Placed 14
     - Example Integration Flow 14
   - User Flow Diagrams 15
     - Feature 1 15
     - Feature ii - Login to your account 16
     - Feature iii - Add/ Edit/ Remove Items in Account 17
     - Feature iv - Deposit Cash Account 18
     - Feature v - Search for Items 19
     - Feature vi - Purchase Item 20
     - Feature vii - View Account Info 21

Feature viii - Manage Inventory 22
Feature ix - Generate Reports 23
Feature x – Interfacing External Stores 24
5. Application-Level Protocol 25
8. Distributed Database Design 26
CockroachDB 26
Schema 27
Partitioning Diagram 27
Database Architecture Diagram 29
7. Initial Time Plan 30
Partitions 30
Phase I Submission 30
Sprint 1 31
Sprint 2 31
Sprint 3 32
Phase II Submission 32
9. Role of Each Member 33

1. Introduction

People and businesses are increasingly involved in the buying and selling process due to rapid growth of digital commerce. The internet has become a vital part of modern economic life, with online marketplaces serving as central hubs for buyers and sellers to conduct transactions from one place or another. This project aims to address the increasing demand for robust and flexible trading platforms by utilizing an online marketplace system that is fully distributed, which was developed as part of the CSE352s Parallel and Distributed Systems course at Ain Shams University's Computer and Systems Engineering Department.

This project's main focus is on developing a distributed marketplace that supports the entire lifecycle of corresponding commercial transactions, from user registration and account management to final purchase and ownership transfer. The system is designed to support multiple users at the same time, ensuring reliability, consistency, and responsiveness throughout the entire process. Its main function is to allow users to create personal accounts, list items for sale at specific prices, deposit funds into accounts linked with them, search for available products, and make purchases that atomically transfer money and item ownership between parties.

Besides its fundamental commercial applications, the system places significant emphasis on the principles of distributed computing. The backend infrastructure is based on low-level TCP socket communication, reflecting a deliberate architectural choice to implement networking at essentially the same level as the main communication layer. This strategy allows for a thorough exploration of the fundamental aspects of distributed systems, such as multi-threading and concurrent client handling while maintaining consistency across all environments. The primary application server is responsible for receiving TCP connections, setting up a distinct thread for each connected client, and handling requests according to an established custom protocol using SMART (structural message format).

Furthermore, the system employs a distributed database model that is powered by CockroachDB, an open-source cloud-based distributed SQL database that offers fault tolerance and horizontal scalability. The database is divided into multiple nodes, with user and item data partitioned by identifier range while transaction information is partitioning according to date range, resulting in optimal query performance due to the balance of load distribution. Apart from the socket-based communication layer, the system provides three selected functions — account creation, user login, and cash deposit – as REST web services through a simple Java HTTP server, fulfilling the requirement of interoperability with standard web service protocols. Due to the inability of browsers to initiate direct TCP connections, a WebSocket bridge server is utilized to facilitate communication between the React-based web frontend and the raw TCF socket backend.

2. Target Benefeciaries

The distributed online marketplace system caters to the following groups of customers:

1. Individual Sellers.

Those who want to sell goods online can create an account, list items at specified prices, manage their inventory, and monitor their sales history and pending listings on the platform.

2. Individual Buyers.

Those interested in buying merchandise can search for items by name or brand, deposit funds into their accounts, and conduct secure transactions with guaranteed transfer of money and item ownership.

3. External Partner Stores.

A dedicated API for the Partner Store facilitates integration with external businesses, enabling third-party businesses to list and manage products on the platform without a user interface.

4. Platform Administrators.

By offering comprehensive platform visibility, the features also facilitate real-time inventory management and transaction reporting, which enables administrators to manage day–to-day operations effectively.

5. Developers and System Integrators.

3. Adopted Programming Language

The system is implemented using the following languages and technologies.

Component	Language / Technology	Framework
Socket Server	Java	None — Raw TCP Sockets
Bridge Server	Java	None — Pure Java
Frontend	JavaScript	React
REST Server	Java	Spring Boot (Bonus Feature)
Database	SQL	CockroachDB

Java was selected as the primary backend language for both the Socket Server and the Bridge Server due to its native support for low-level socket programming through the java.net package, as well as its strong multi-threading capabilities that allow handling multiple concurrent client connections efficiently.

No frameworks are used in any core backend component, in direct compliance with the project requirement that the main application must be implemented using low-level socket communication only.

React was chosen for the frontend to deliver a dynamic and responsive user interface, enabling smooth interaction with all marketplace's features across both the WebSocket and REST communication channels.

Spring Boot was adopted exclusively for the bonus REST web service layer, where framework usage is permitted, providing a clean and efficient way to expose the three selected features as standard REST endpoints.

4. System Architecture

Overview

This system is a distributed three-tier architecture consisting of a presentation tier, an application tier, and a data tier.

The **Presentation tier**, implemented using React, is the layer with which clients interact. It houses the user interface and communicates with the application tier through two channels: a WebSocket connection for all core marketplace features, and HTTP REST calls for the three bonus web service features.

The **Application tier** is the heart of the system and contains all business logic. Unlike a traditional monolithic backend, the application tier is divided into three distinct components to satisfy the requirement of low-level socket communication while maintaining a clean and organized structure.

The **Socket Server**, implemented in pure Java with raw TCP sockets and no frameworks, serves as the core of the system and handles all nine marketplace features. The **Bridge Server**, also implemented in pure Java, acts as a protocol translator between the React frontend's WebSocket connection and the Socket Server's raw TCP interface, since browsers are inherently unable to open direct TCP connections.

The **Spring Boot REST Server** is implemented as a bonus component and exposes three selected features as standard REST web service endpoints, communicating with the Socket Server via TCP to execute the required logic without duplicating any business logic.

The **Data tier** contains all the data needed for the system. It is powered by CockroachDB, a distributed SQL database, and communicates with the Socket Server through the JDBC API. The database is partitioned across multiple nodes to ensure horizontal scalability and fault tolerance.

High Level Architecture Diagram

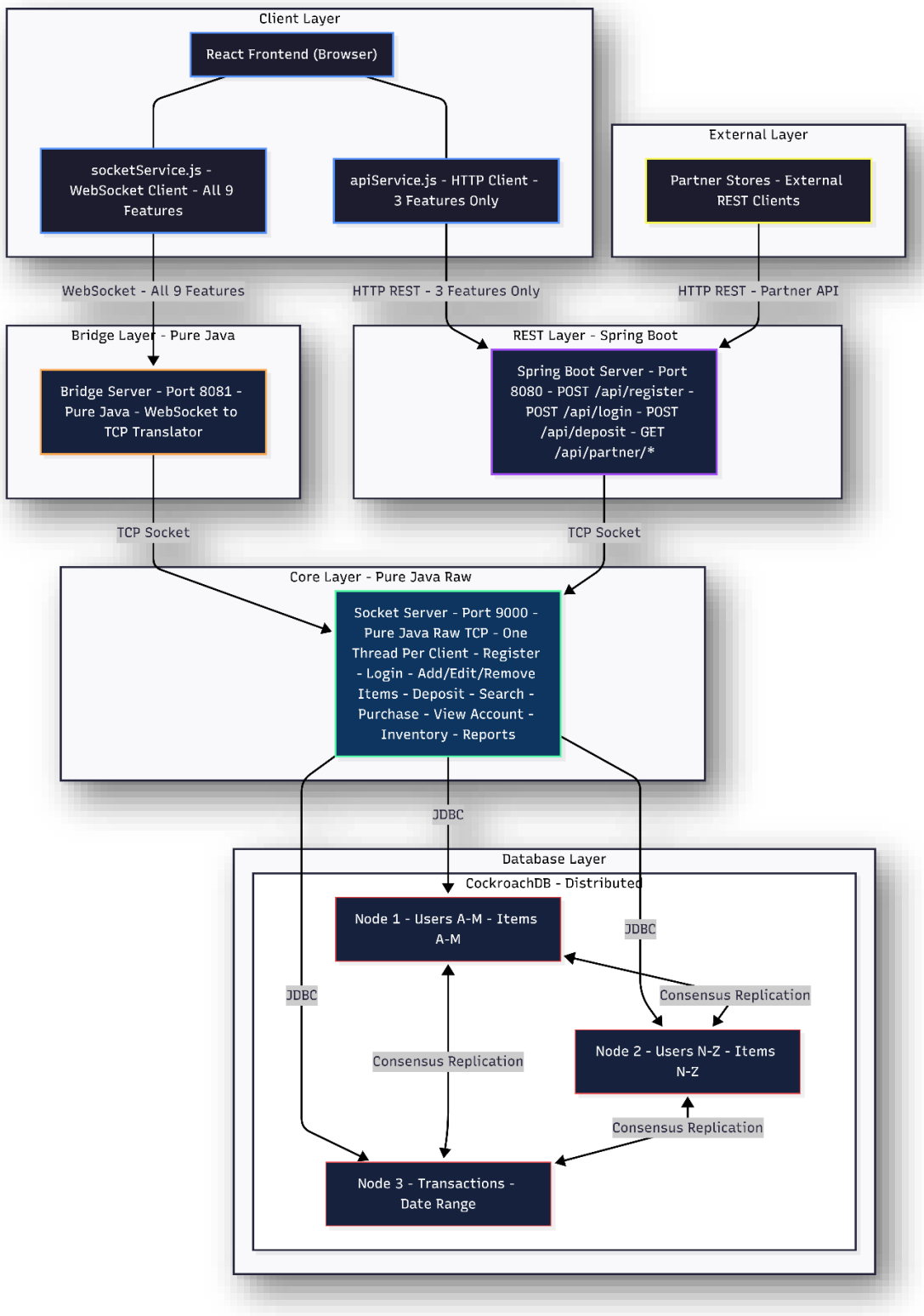


Figure 1: High Level System Architecture

Detailed Backend Layout

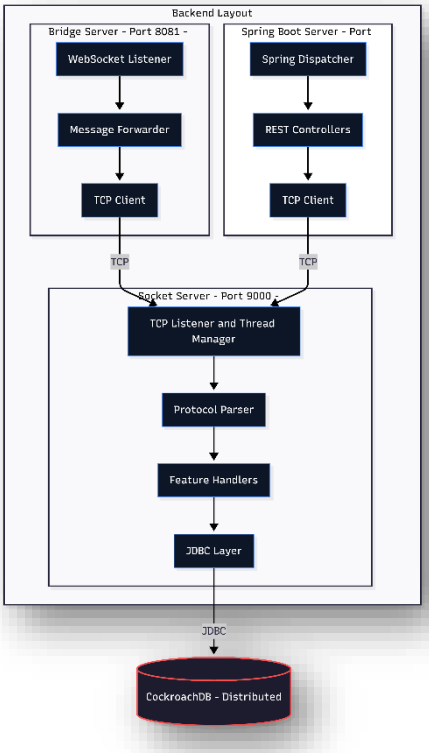


Figure 2: Backend Higher level Architecture

Internal Modules

The backend is structured into three separate components, each with a distinct responsibility and internal organization.

Socket Server

The Socket Server is the core of the entire system. It is implemented using pure Java raw TCP sockets with no frameworks, in direct compliance with the project requirement for low-level socket communication. It is internally structured into handlers and services to achieve separation of concerns.

When a client connects, the server spawns a dedicated thread to handle that connection, allowing multiple clients to be served concurrently. The Protocol Parser reads the incoming message in the format COMMAND|{JSON}, identifies the command, and routes the request to the appropriate handler. Each handler delegates the business logic to its corresponding service, which then interacts with CockroachDB via JDBC.

- The **Protocol Parser** reads raw TCP messages and routes them to the correct handler.
- The **Handlers** receive parsed requests and delegate to the service layer.
- The **Services** contain all business logic, validate conditions, and coordinate with the database.

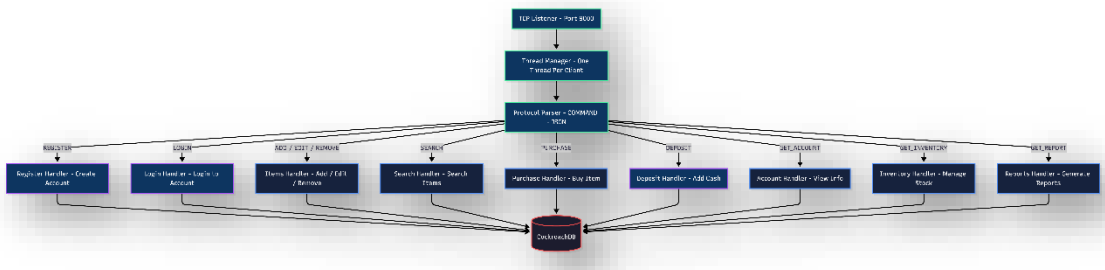


Figure 3: Socket Server Internal Architecture

Bridge Server

The Bridge Server is implemented in pure Java and contains zero business logic. Its sole responsibility is to maintain a WebSocket connection with the React frontend and forward all messages to the Socket Server over TCP, then relay the responses back to the browser. It exists purely to bridge the gap between the browser's WebSocket protocol and the backend's raw TCP protocol.

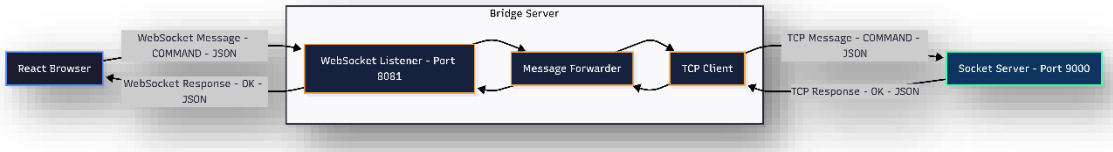


Figure 4: Bridge Server Internal Architecture

Spring Boot REST Server

The Spring Boot REST Server is implemented as a bonus component and exposes three marketplace features as standard REST endpoints. Each controller receives an HTTP request, builds the equivalent TCP protocol message, forwards it to the Socket Server, receives the TCP response, and returns it as a clean HTTP JSON response to the caller. This design ensures that all business logic remains exclusively in the Socket Server, and the REST server acts purely as an additional entry point into the same core logic.

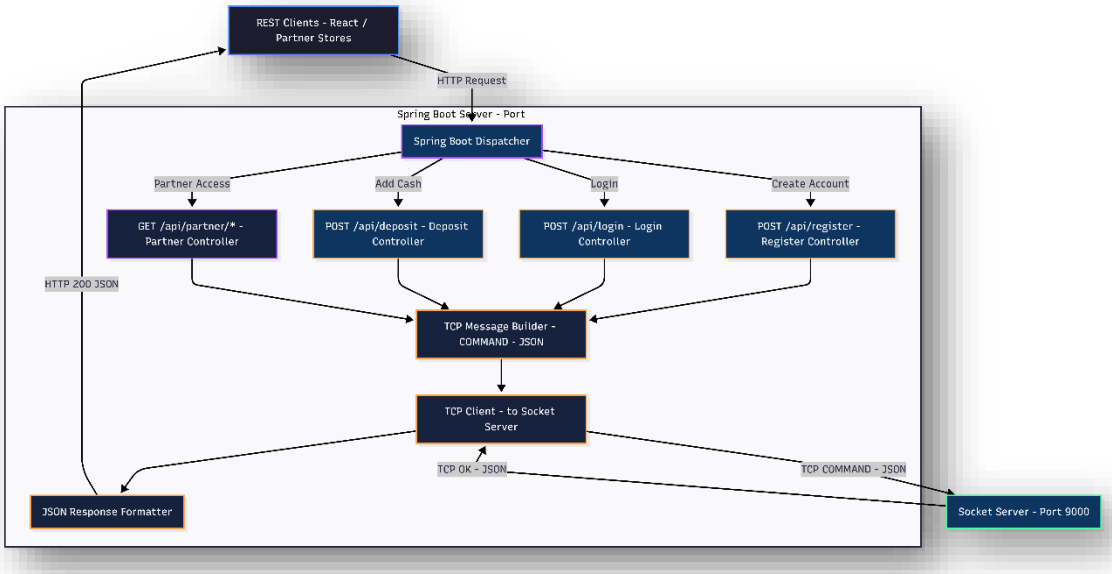


Figure 5: Spring Boot REST Server Internal Architecture

React Frontend

The React frontend is internally organized into screens and service modules. The socketService.js module manages the WebSocket connection to the Bridge Server for all nine core marketplace features. The apiService.js module handles HTTP REST

calls to the Spring Boot server for the three bonus features. This separation ensures that each communication channel is managed independently and cleanly.

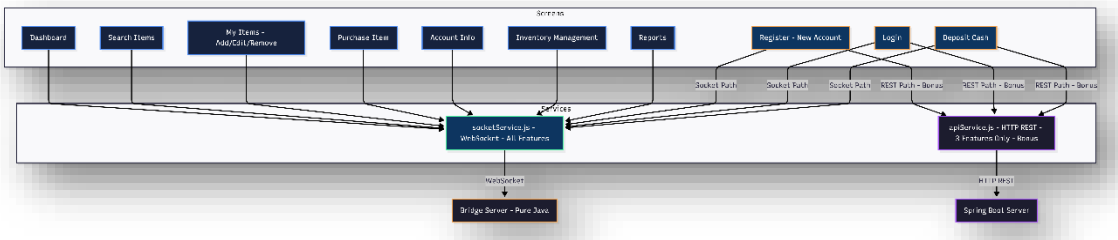


Figure 6: React Frontend Internal Architecture

Shared Models

Common models are centralized and shared across backend components to avoid redundancy and improve consistency. They represent the core business objects of the system stored in the database.

The design rule is that any model used by multiple modules is placed in the shared models, while models used by only one module remain local to that module.

Shared Entities:

- User
- Item
- Transaction
- Inventory

Shared DTOs:

- UserResponseDto
- ItemDto
- TransactionDto

DTOs are used to transfer structured data between backend components and the frontend without directly exposing internal entity classes.

Custom Exceptions:

- InsufficientBalanceException
- ItemNotFoundException
- UserNotFoundException
- UnauthorizedActionException

These custom exceptions provide clear and reusable error handling across all backend components.

Three Features Implemented as REST (Bonus)

Register was chosen because it is the entry point of the entire system, and every other feature depends on having an account. It demonstrates a complete web service cycle: receiving, validating, and processing input, then returning a structured response. It requires no prior state or authentication, making it the simplest possible standalone web service.

Login was chosen because it demonstrates token-based authentication, which is a core distributed systems concept directly relevant to this project. It shows how stateless authentication works across a distributed environment — the server issues a token and never needs to store session state, which is fundamental to scalability.

Deposit was chosen because it demonstrates a transactional web service that validates authorization via JWT, performs a state-changing operation on the database, records an audit trail in the transactions table, and returns a confirmation. This covers more web service concepts than a simple read operation and directly demonstrates distributed database consistency in action.

REST Implementation

The three bonus features are implemented using REST via Spring Boot, which provides native REST support through standard HTTP. Data is exchanged in JSON format, which is lightweight, human readable, and integrates directly with the React frontend without any transformation layer.

SOAP was considered but rejected because it requires a separate Spring-WS dependency, a WSDL file describing every operation, XML schemas for every request and response, and a completely different controller model. Every request and response must be wrapped in a heavy XML envelope, and the React frontend cannot consume SOAP directly without an additional translation layer. This adds complexity that is not justified given that the system does not require any of SOAP's specific advantages.

Benefits of This Architecture

- **Security:** The React frontend has no direct database access. All data flows through the Socket Server, which acts as a mandatory gateway for authentication, authorization, and input validation.
- **Compliance:** The core application is built entirely on raw TCP sockets with no frameworks, satisfying the project requirement for low-level socket communication.
- **Testability:** Each tier and each internal component can be tested in isolation, allowing failures to be precisely located without ambiguity.
- **Modularity:** Separating the frontend, bridge, socket server, REST server, and database into distinct components makes the system easier to organize and understand. Within the backend, the handler and service structures further improve modularity.
- **Maintainability:** Handlers and services each have a specific responsibility, making the code easier to maintain and extend as the system grows.
- **Reusability:** The shared models package allows common entities, DTOs, and exceptions to be reused across multiple components without duplication.
- **Scalability:** CockroachDB supports horizontal scaling and distributed storage, making the data tier more scalable than a traditional single-node database.

API for Interfacing External Stores

Overview

This API allows external stores to integrate with our marketplace. It enables partner stores to browse available products and place orders on behalf of their customers. All communication is over HTTP and all responses are in JSON format.

Authentication

All endpoints require an API key. Each external store is issued a unique API key upon registration with the marketplace. The API key must be included in every request as an HTTP header.

X-API-KEY: your_api_key_here

If the API key is missing, invalid, or belongs to an inactive store, the request will be rejected immediately with a 401 response before reaching any endpoint.

Endpoints

1. **Get Available Products**
   Retrieves a list of all items currently available for sale on the marketplace.

   GET /api/store/products

   Request Headers

   X-API-KEY: your_api_key_here

2. **Place an Order**
   Places a purchase order on behalf of an external store's customer. Upon success, the marketplace seller's balance is credited, the item is marked as sold, and a transaction record is created referencing the external store.

POST /api/store/order

Request Headers

X-API-KEY: your_api_key_here
Content-Type: application/json

Request Body

```
json
{
  "item_id": 1,
  "customer_reference": "CUST-123",
  "customer_email": "customer@example.com"
}
```

Field Descriptions

Field	Type	Required	Description
item_id	Long	Yes	The ID of the item to purchase
customer_reference	String	Yes	The external store's own customer identifier
customer_email	String	Yes	The email of the customer placing the order

What Happens When an Order is Placed

When a successful order is placed the following happens automatically in one atomic operation:

1. API key is validated
2. Item is checked to confirm it is still available
3. Seller's balance is increased by the item price
4. Item status is updated to SOLD
5. Transaction is recorded with the external store ID, seller ID, item ID, amount, customer reference, and timestamp
6. Transaction receipt is returned to the external store

If any of these steps fail, the entire operation is rolled back and no changes are made to the database.

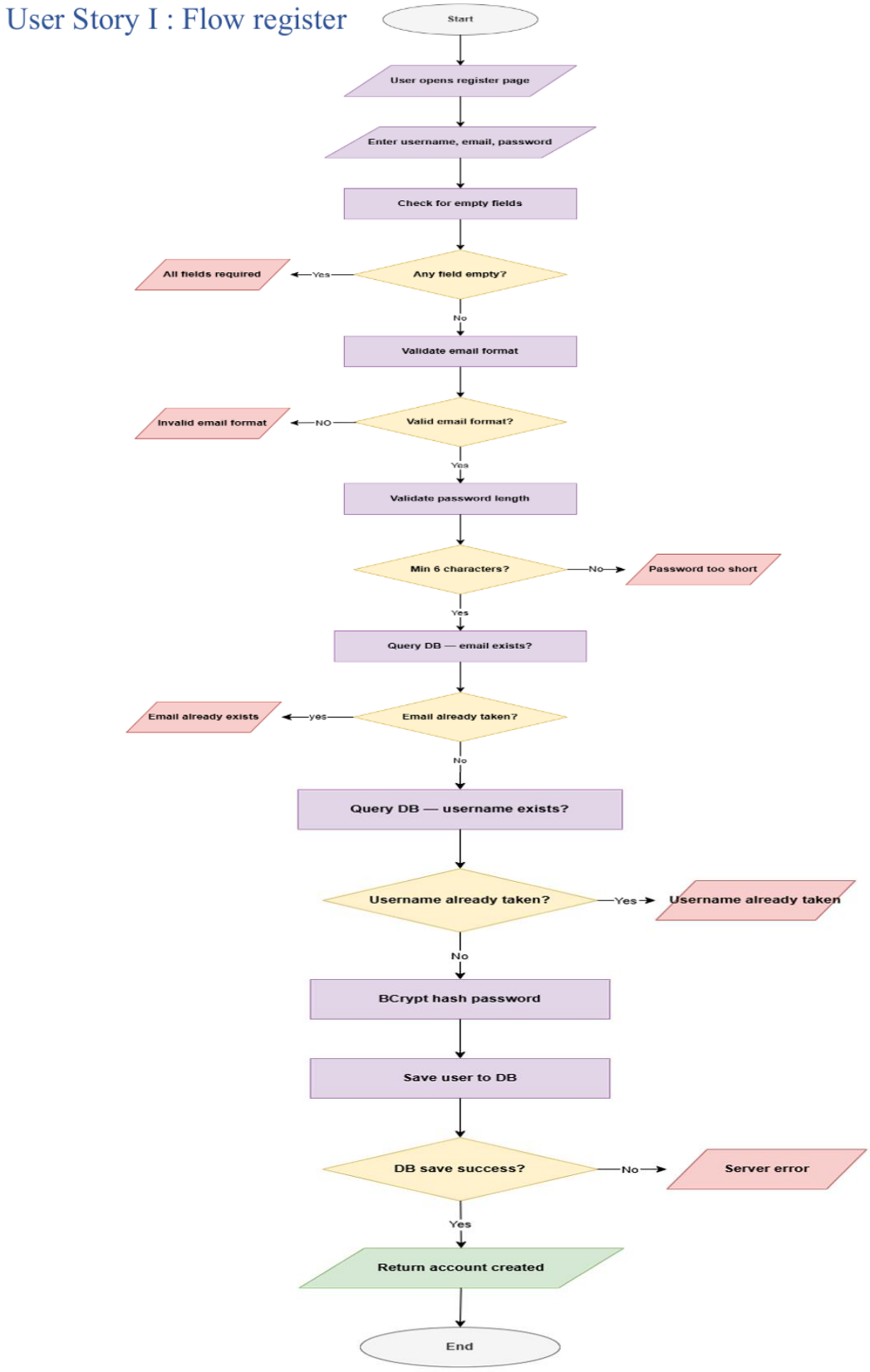
Example Integration Flow

A typical external store integration works as follows.

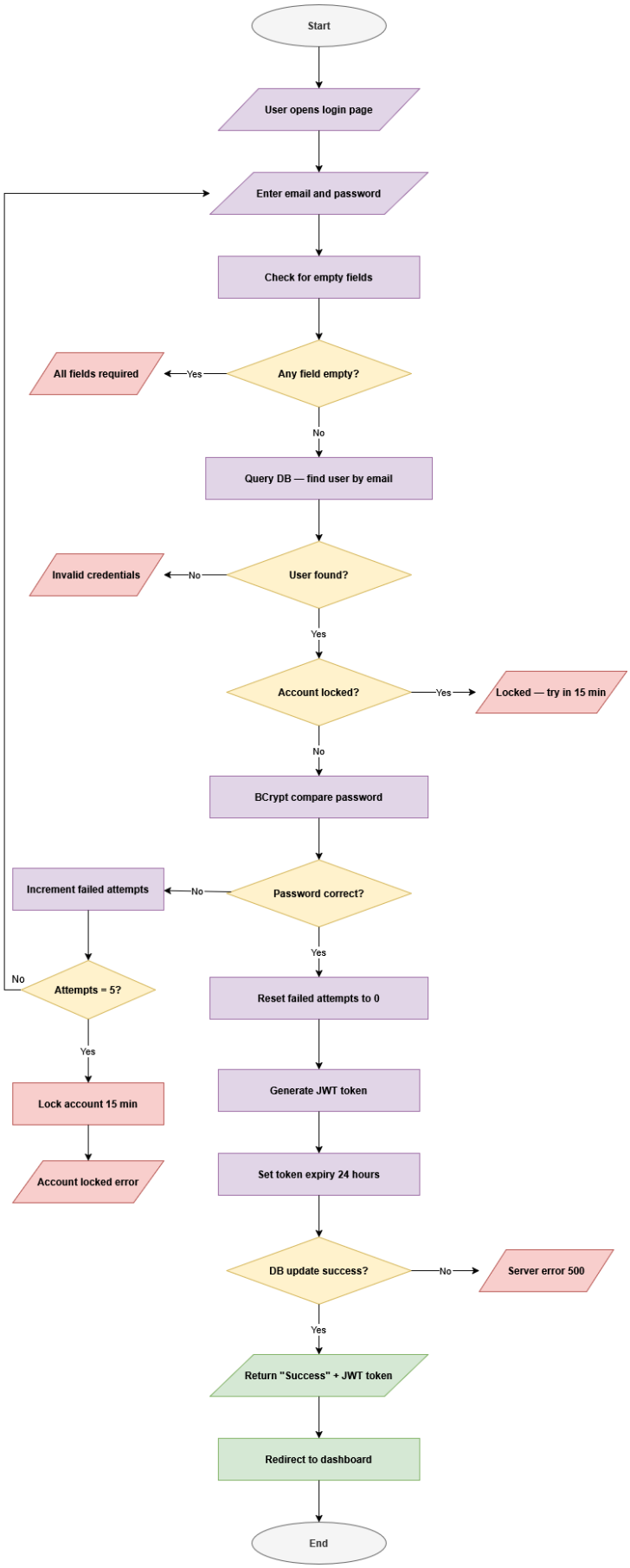
1. The store calls GET /api/store/products to fetch and display available items to their customers.
2. When a customer selects an item and confirms purchase, the store calls POST /api/store/order with the item ID and their customer reference.
3. The marketplace processes the sale and returns a receipt. The store stores the transaction ID from the receipt for their own records and confirms the order to their customer.

User Flow Diagrams

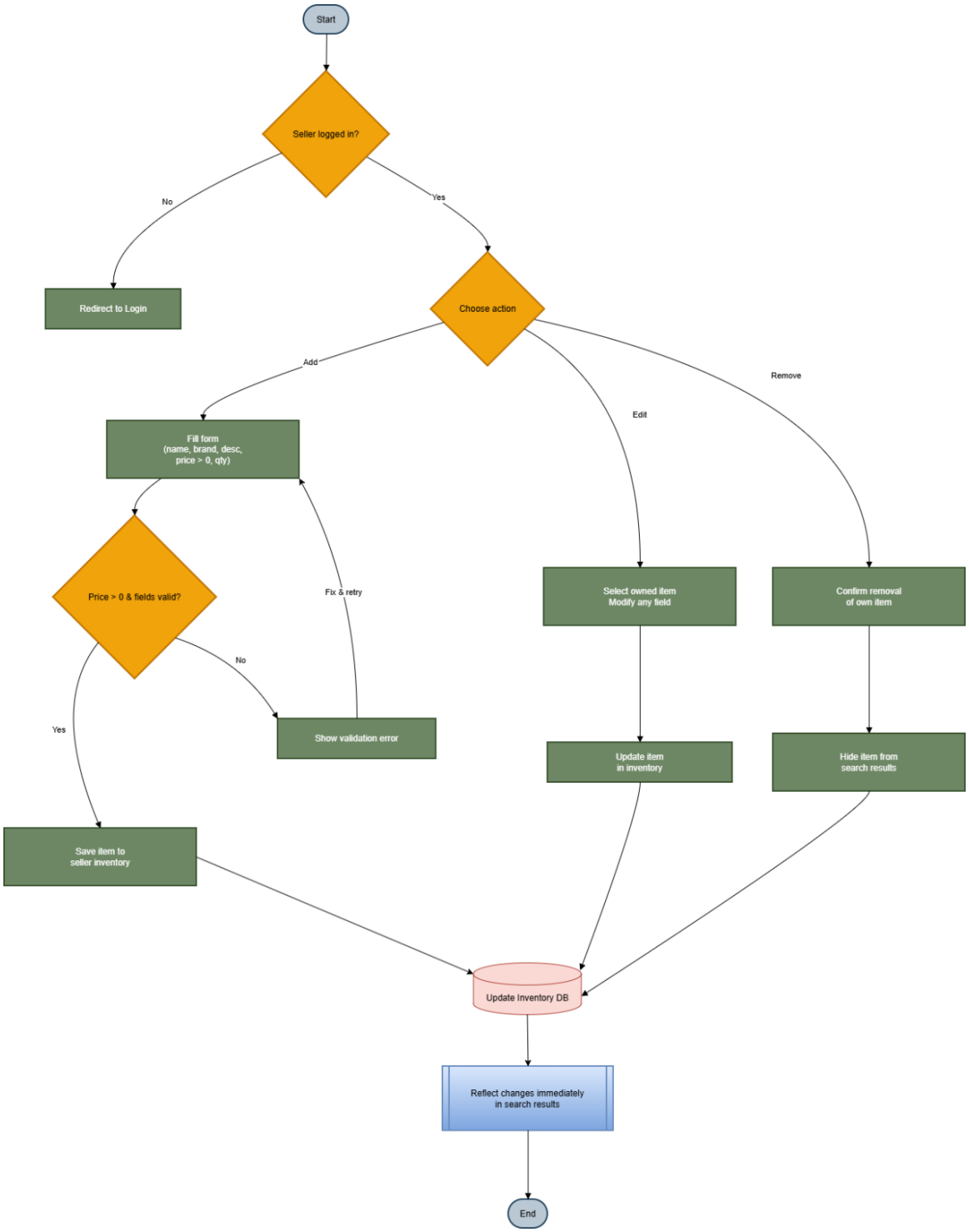
Feature 1



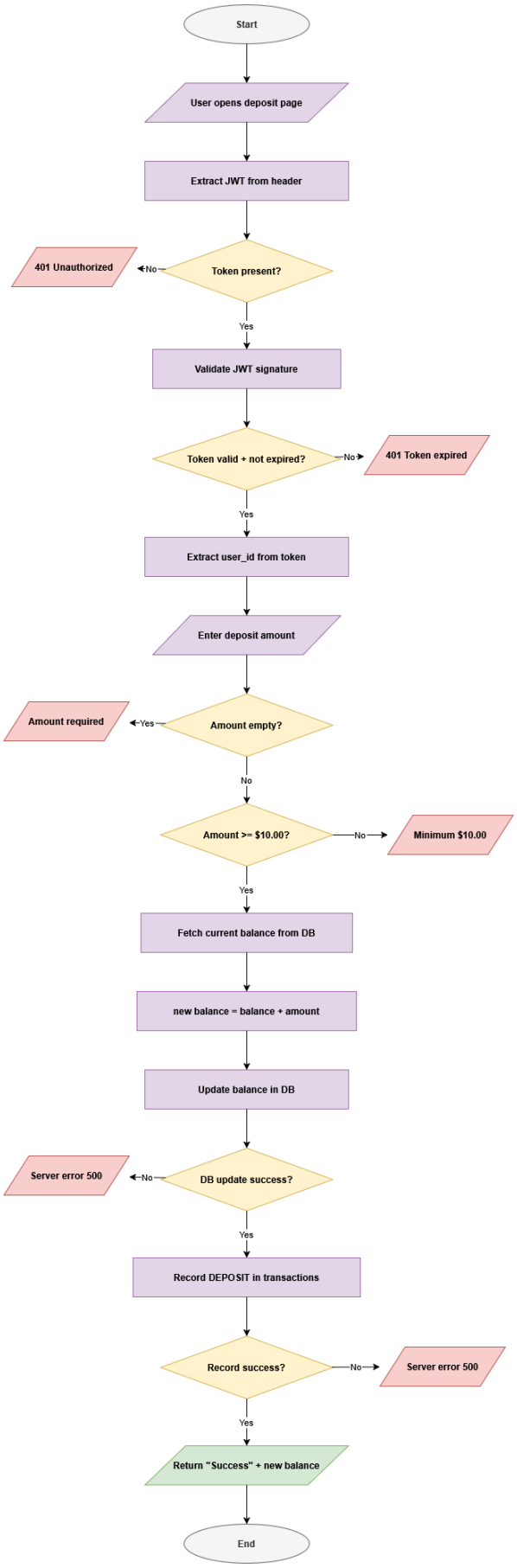
Feature ii - Login to your account



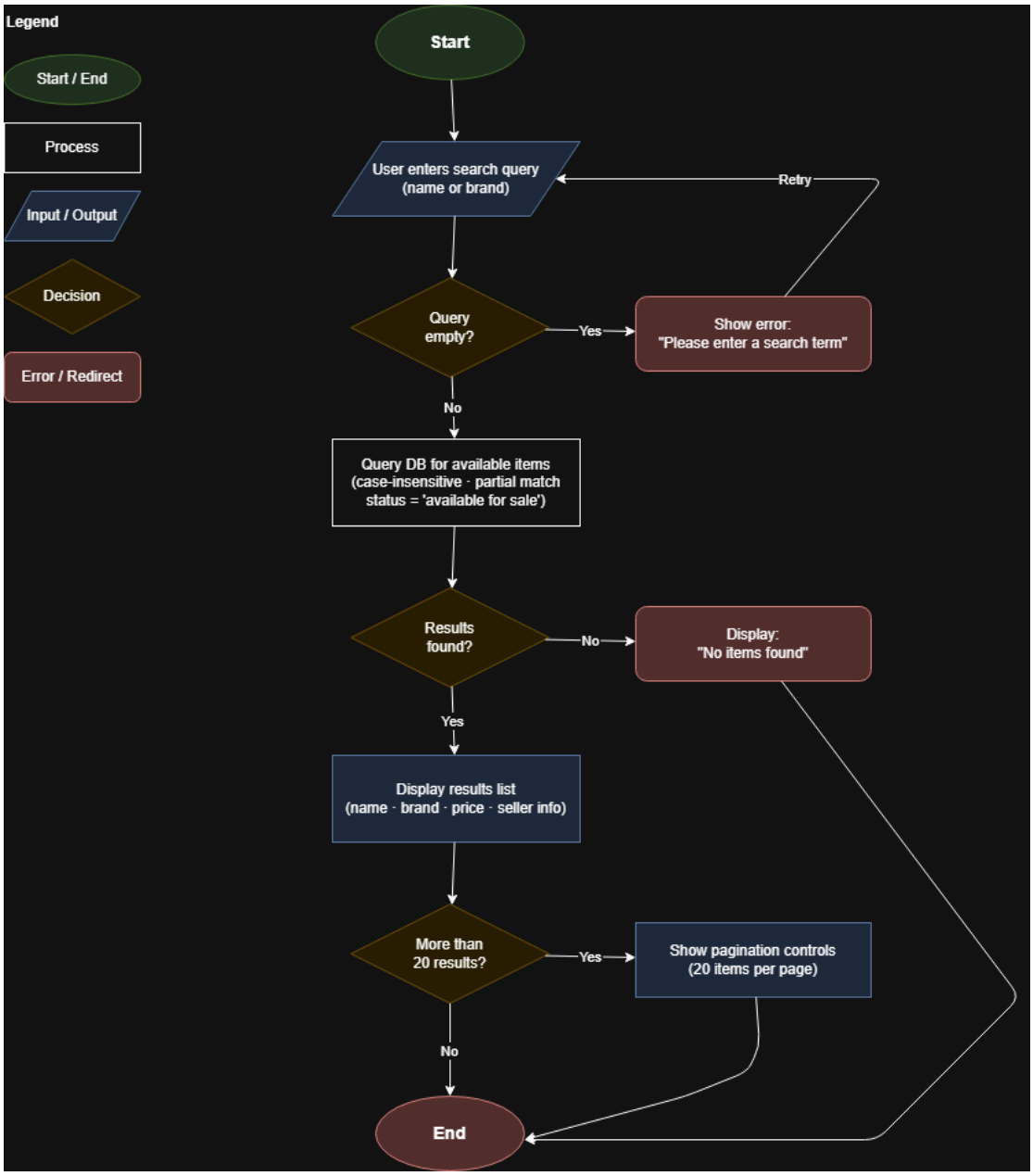
Feature iii - Add/ Edit/ Remove Items in Account



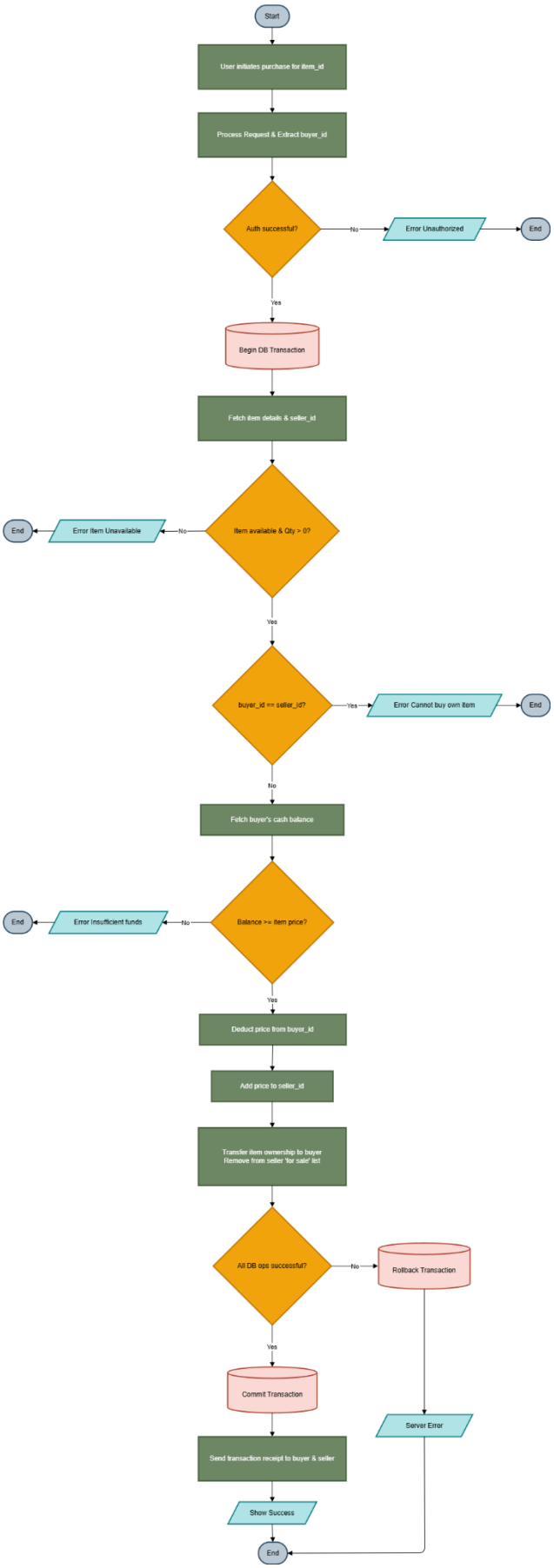
Feature iv - Deposit Cash Account



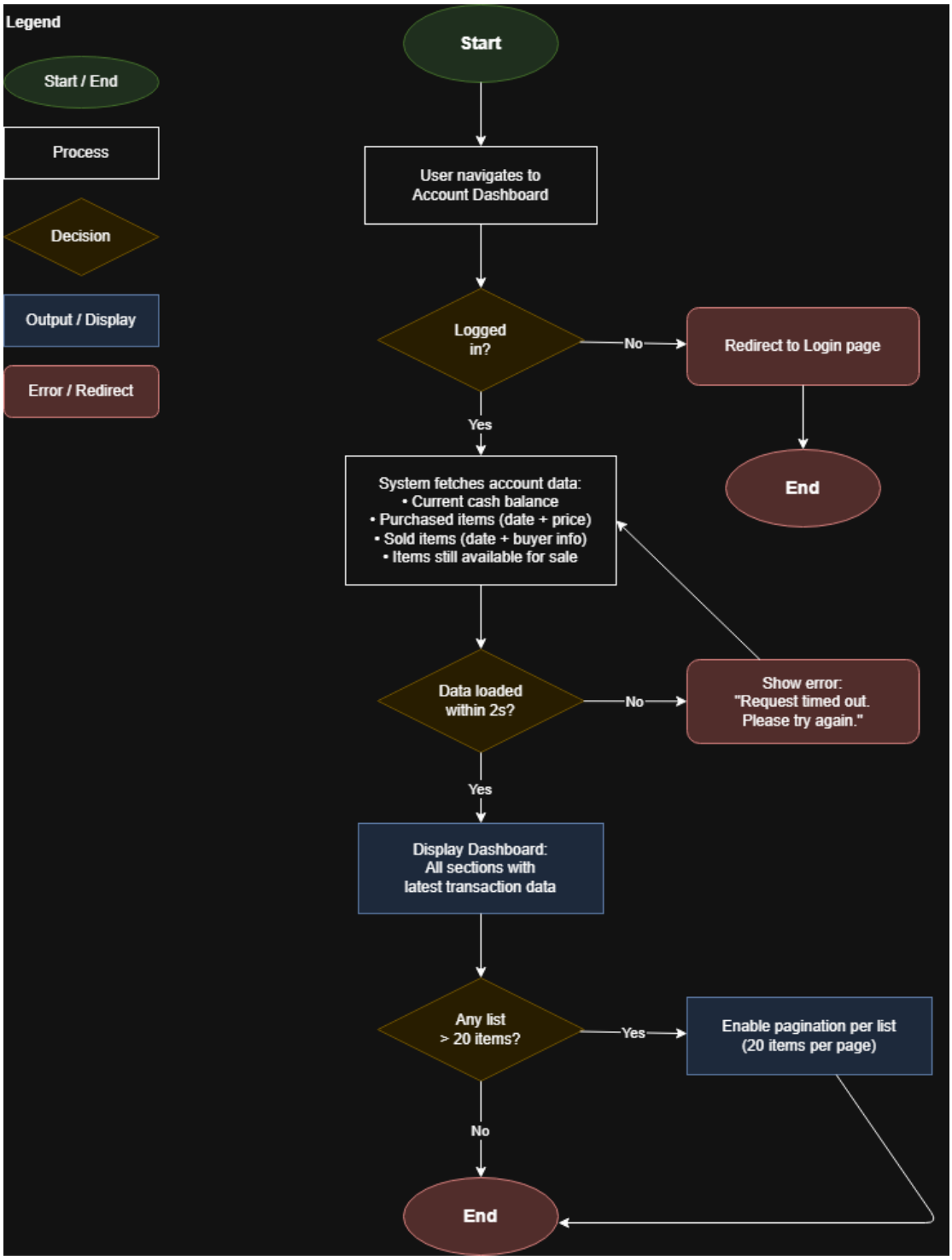
Feature v - Search for Items



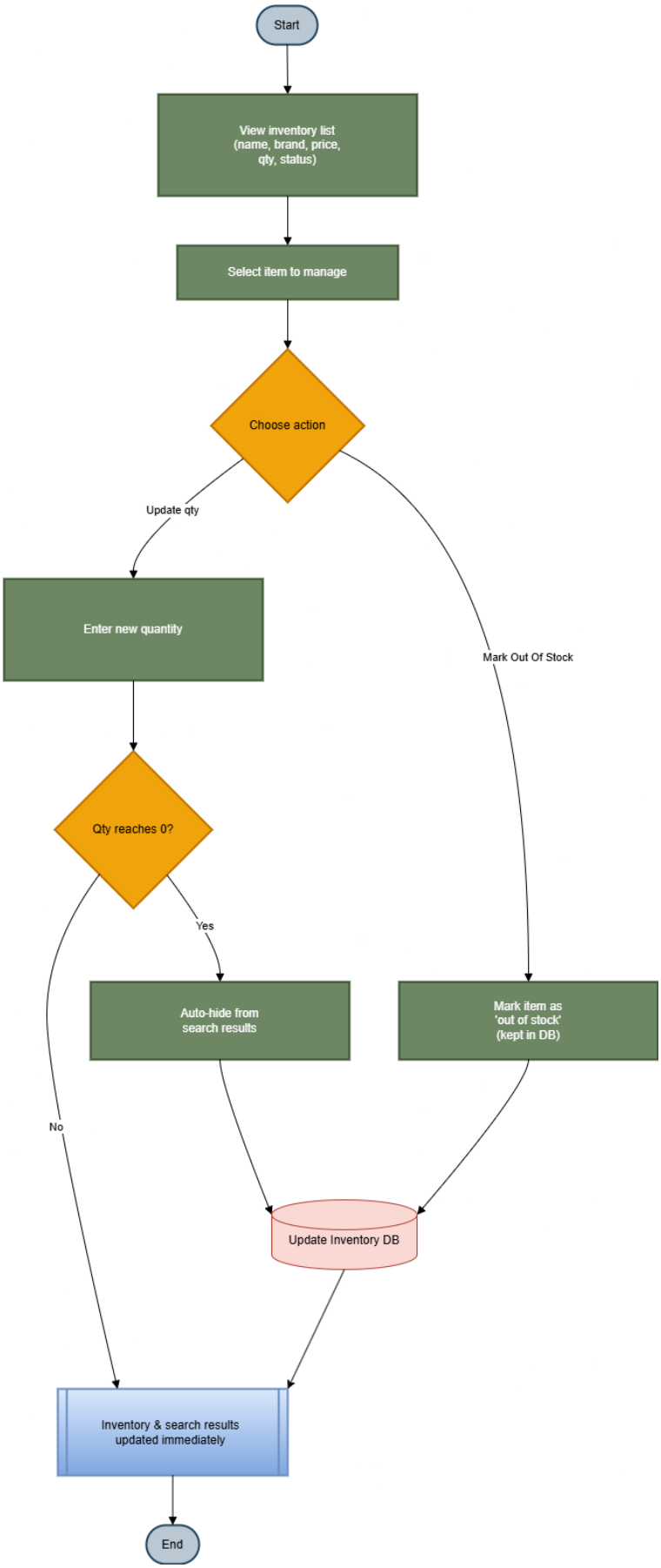
Feature vi - Purchase Item



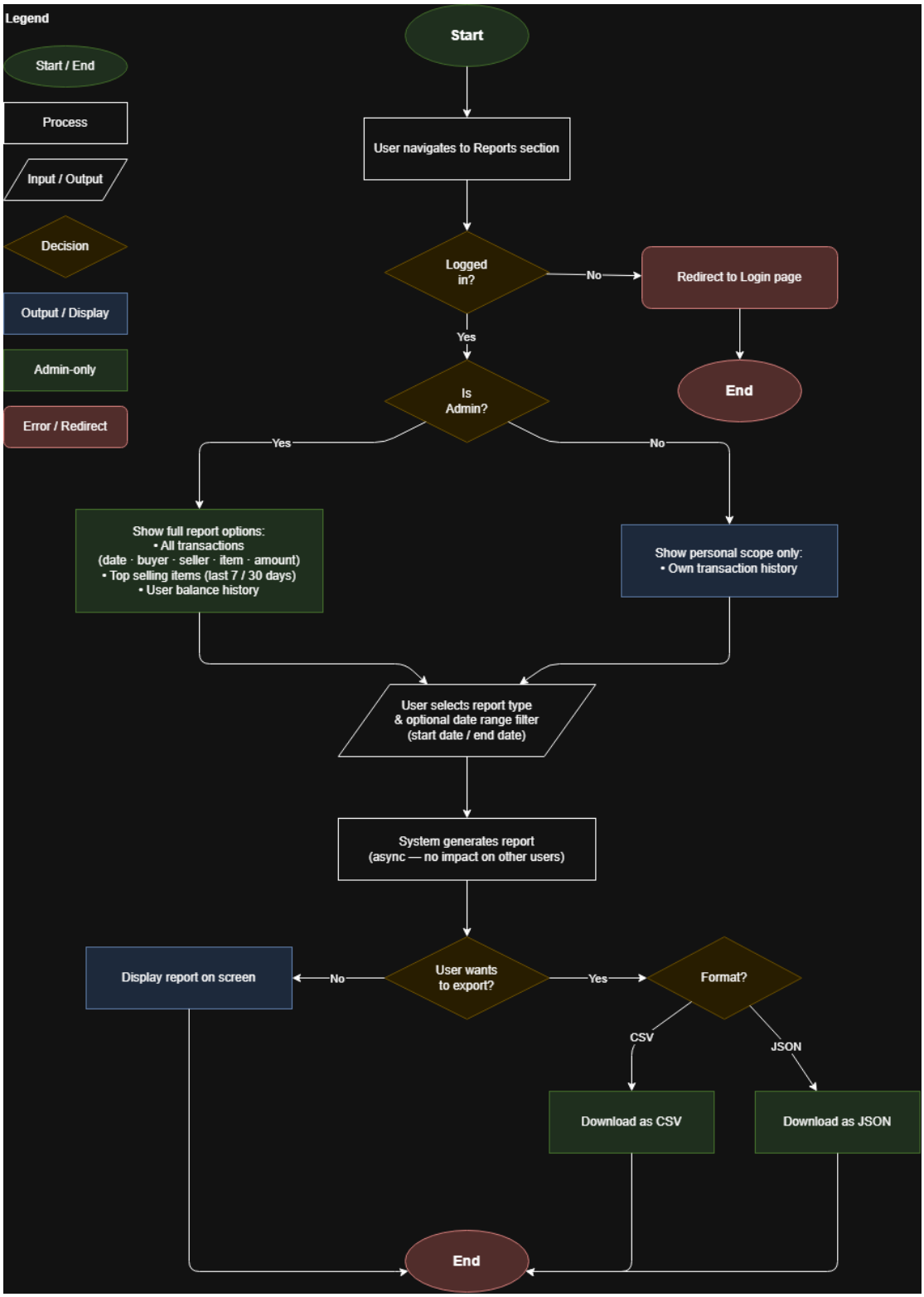
Feature vii - View Account Info



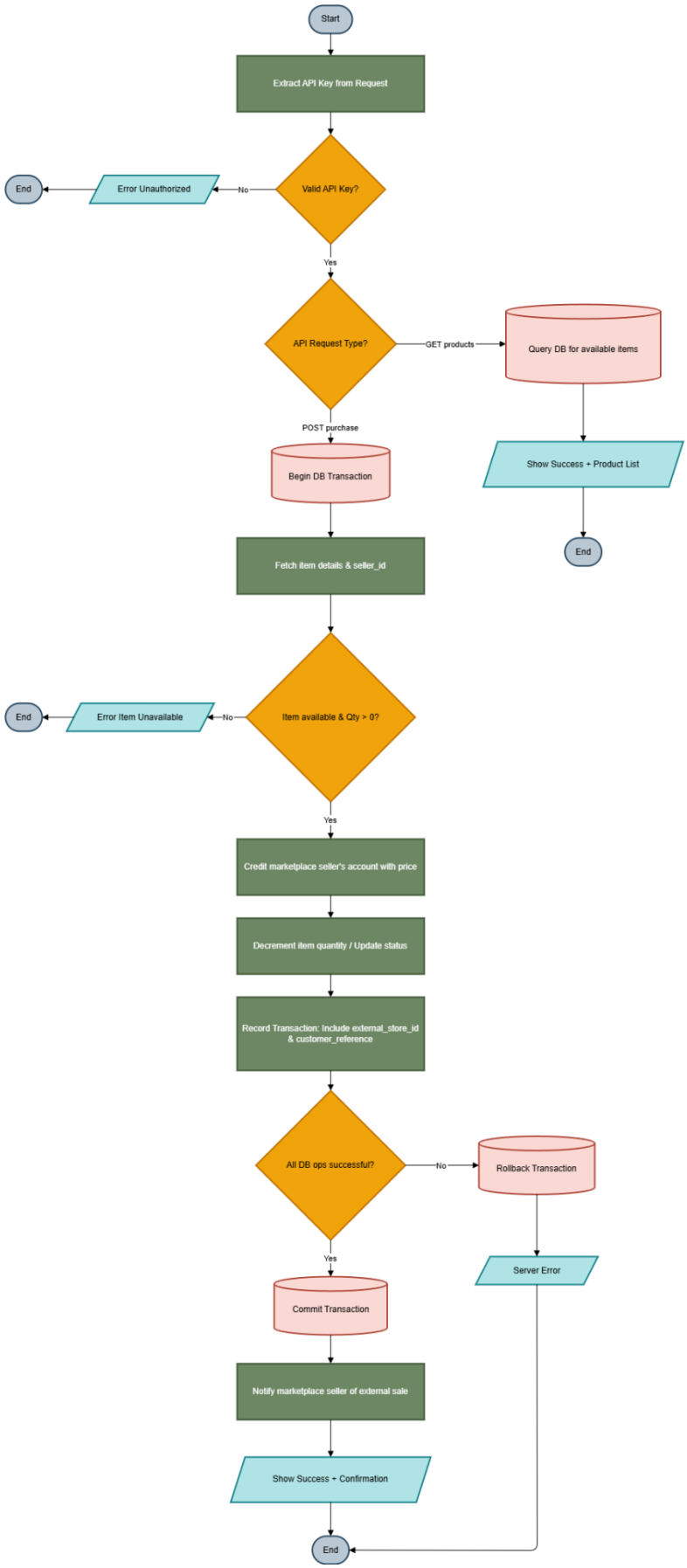
Feature viii - Manage Inventory



Feature ix - Generate Reports

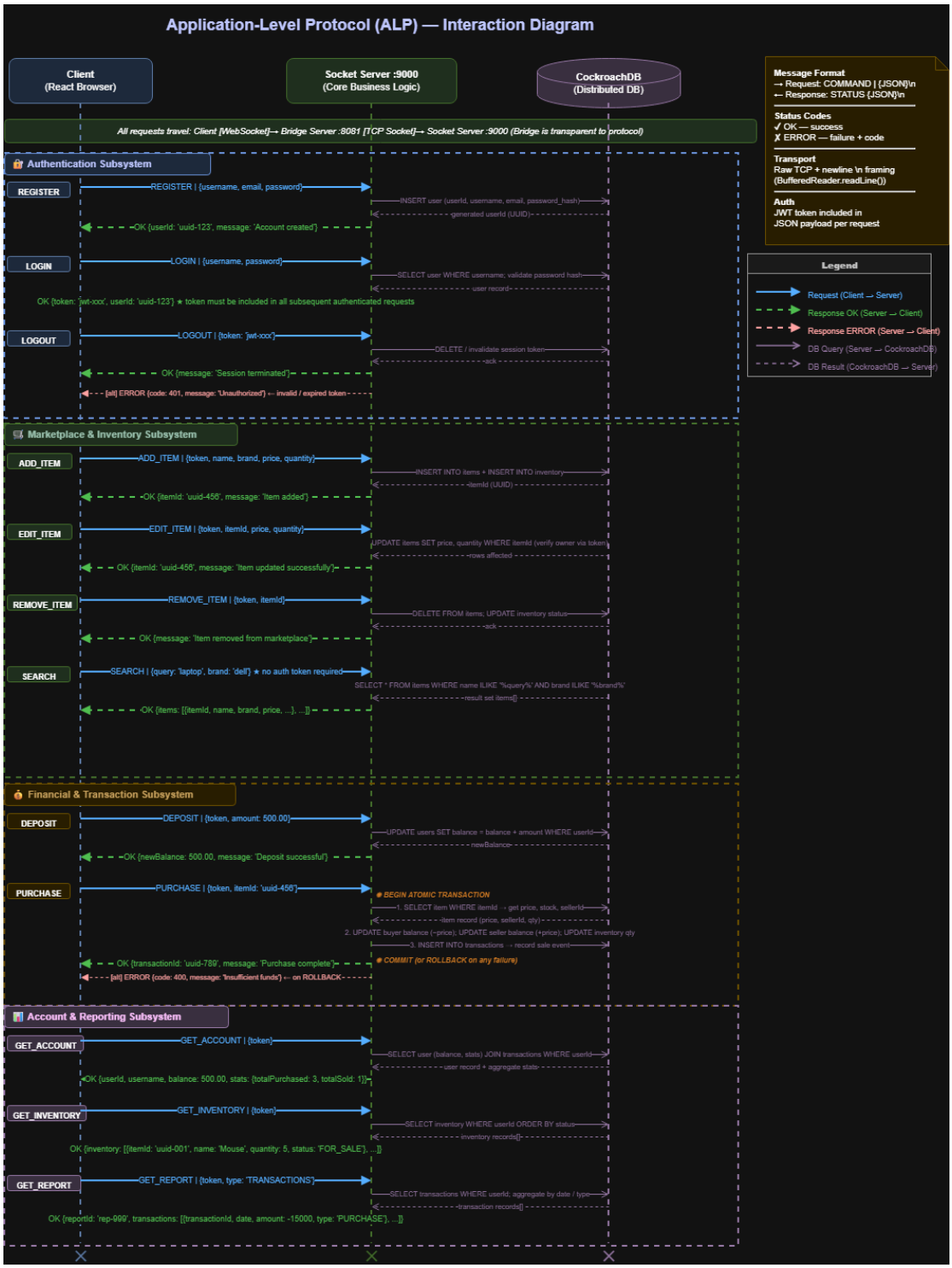


Feature x – Interfacing External Stores



5. Application-Level Protocol

Since the core communication uses raw TCP sockets, a custom application-level protocol has been defined for this system. This protocol relies on clear-text commands combined with JSON payloads to facilitate structured data exchange between the client and the core marketplace server.



8. Distributed Database Design

CockroachDB

CockroachDB is a distributed SQL database that is designed for cloud applications. It works like traditional SQL (PostgreSQL-style) but scales horizontally.

Key Benefits

1. Horizontal Scalability

- Easily scale by adding more nodes.
- No need to upgrade a single powerful server.
- Handles large-scale systems smoothly.

2. High Availability

- Data is automatically replicated across multiple nodes.
- If one node fails, system keeps running.
- Built-in fault tolerance.

3. Distributed Architecture

- Data is spread across different machines/regions.
- Enables global applications with low latency.

4. Fault Tolerance & Self-Healing

- Automatically recovers from failures.
- Rebalances data without manual intervention.

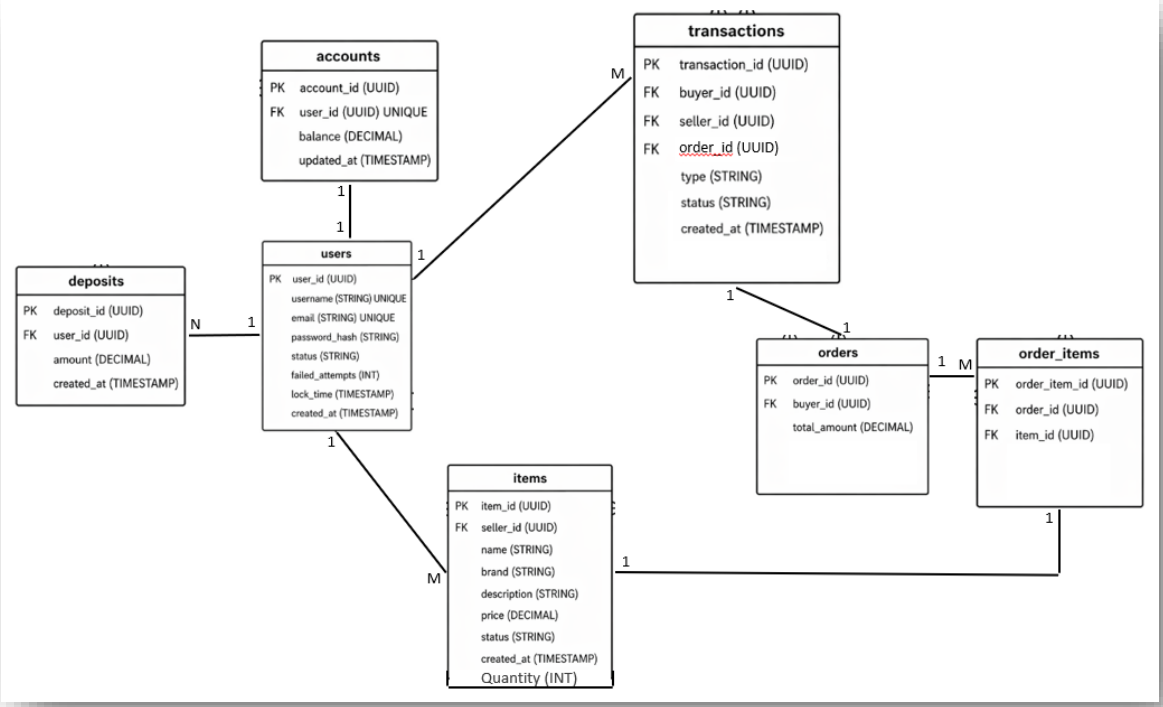
5. SQL Support (Easy to Learn)

- Uses familiar SQL (like PostgreSQL)
- Easy for developers to adopt.

6. Geo-Distribution

- Store data closer to users worldwide.
- Improves performance and user experience.

Schema



Partitioning Diagram

Nodes (3)

nodes	Uptime	Replicas	Capacity Usage	Memory Use	vCPUs	Version	Status
localhost:26257 (n1)	41 minutes	56	0 %	2 %	12	v25.3.7	LIVE
localhost:26259 (n2)	40 minutes	56	0 %	2 %	12	v25.3.7	LIVE
localhost:26258 (n3)	40 minutes	56	0 %	2 %	12	v25.3.7	LIVE

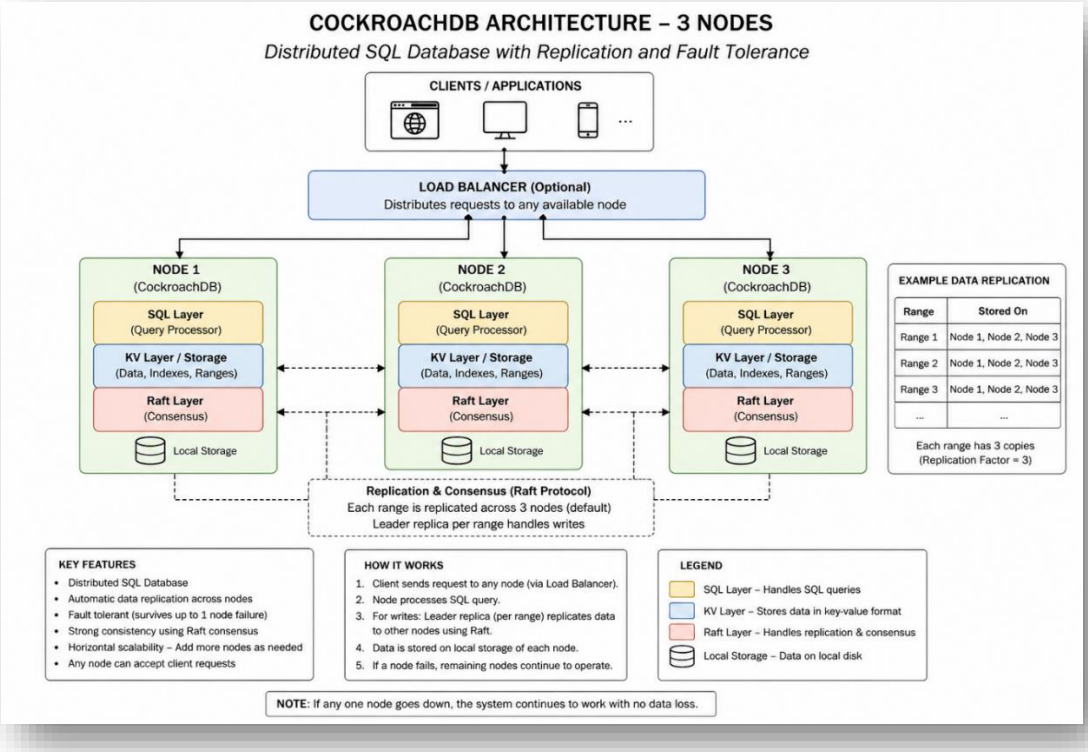
```
root@localhost:26257/marketplace> SHOW RANGES FROM TABLE USERS;
   start_key    |        end_key        | range_id | replicas | replica_localities | voting_replicas | non_voting_replicas | learner_replicas | split_enforced_until
----------------+-----------------------+----------+----------+--------------------+-----------------+---------------------+------------------+----------------------
  …/<TableMin> | <after:/Table/108/6> |       75 | {1,2,3}  | {"","","}          | {1,3,2}         | {}                  | {}               | NULL
(1 row)

Time: 15ms total (execution 15ms / network 1ms)
```

- Each table may be split into many ranges of rows each of 64MB or less according to traffic load.
- Each range is replicated in the 3 nodes.

- Automatically number of ranges (Chunks) are dynamically managed by cockroach DB itself according to traffic load or failed nodes.
- Horizontal partitioning
- Shard key is the primary key of each table so that avoid hot nodes to balance the traffic.

Database Architecture Diagram



- KV is shortcut for key value indices storage for fast search.
- Raft layer manages the replications between nodes in which the two nodes must agree for the update or insertion for successful query

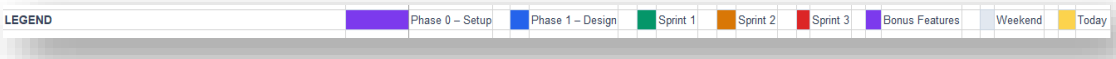
7. Initial Time Plan

Partitions

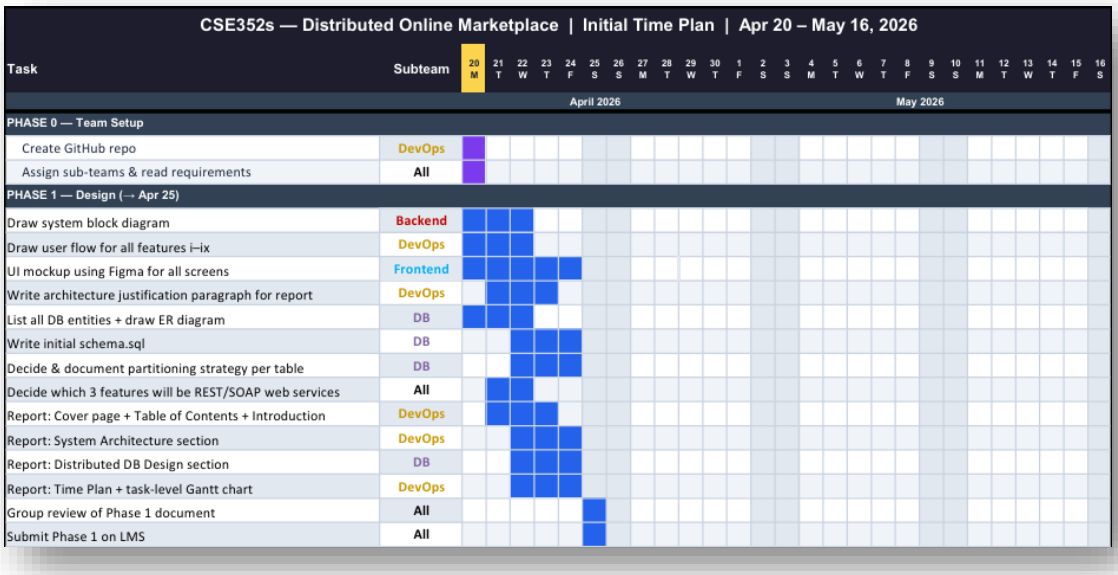
The project would be split into:

1. Pre-phase I planning and tasks.
2. Three sprints for the project itself.
3. Final tasks before phase II submission.

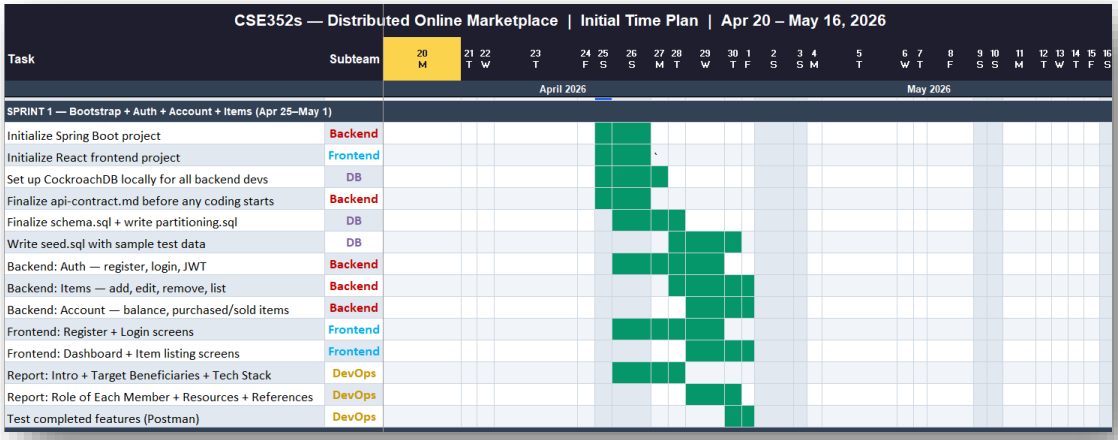
And all the charts are based on this legend:



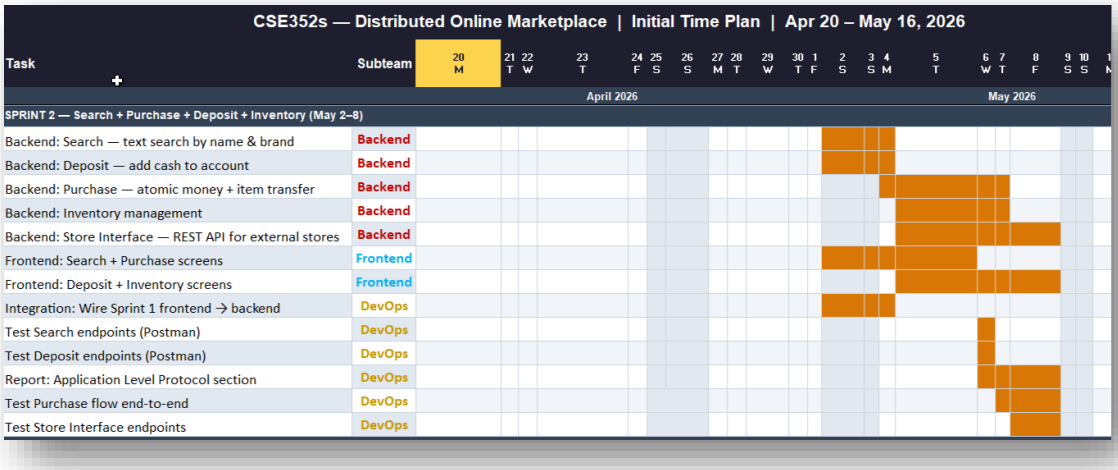
Phase I Submission



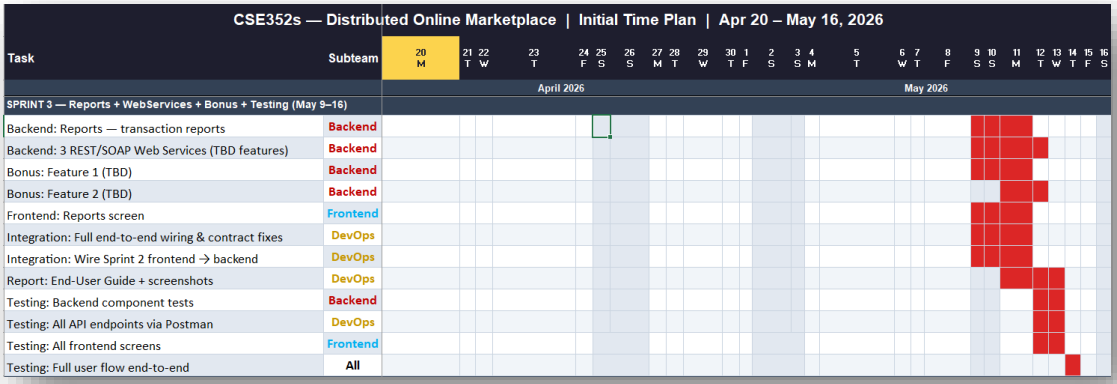
Sprint 1



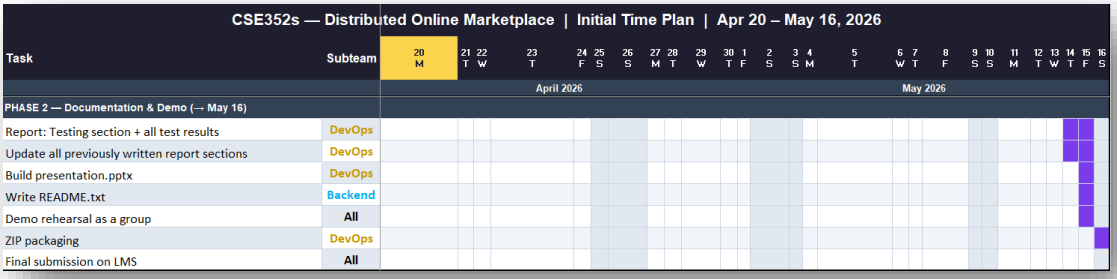
Sprint 2



Sprint 3



Phase II Submission



9. Role of Each Member

Member name	ID	Role
Hazem Mohamed Salah ElDin	2300774	Backend
Mathew Mina Youssef Aziz	2300910	Backend
Marten Amgad Zakaria	2200751	Backend
Ziad Ahmed Orabi	2200399	Backend
Julia Nasser Mosaad	2200760	Front End
Maria Karam Albert	2200593	Front End
Kerlos Melad Fakhry	2200763	Database
Andrew Ehab Tharwat	2200249	Integration/devOps
Arsany Hany Anwar	2200655	Integration/devOps

Backend - Socket

- Design the application-level protocol (ALP)
- Design the API contract for web services.
- Implement all 9 core features over TCP sockets.
- Handle atomic transaction integrity for purchases.
- Handle multi-threaded client connections.
- Integrate with the database layer via JDBC.

Backend - Web Services

- Implement at least three features as REST web services.
- Implement Partner Store API; an interface allowing external stores to sell the marketplace's products.
- Integrate with the database layer.

Frontend

- Build all UI screens for user-facing features.
- Create mock UI.
- Connect to socket server via WebSocket through the bridge.

- Connect to Spring Boot via HTTP REST for the three web service features.
- Own end-to-end user flow coherence.

Database

- Design the full schema (Users, Items, Transactions, Inventory, etc.)
- Decide and implement the partitioning strategy.
- Deliver SQL files.
- Provide a comprehensive documentation.

DevOps/Integration

- Manage GitHub: branching strategy, PRs, protect main and dev.
- Organize the report and presentation.
- Create and update the time plan.
- Build the WebSocket-to-TCP bridge server.
- Wire frontend to backend as features complete
- Catch integration errors early.
- Own demo session preparation.